

Dimensionality Reduction

AIE324 — Unsupervised Machine Learning

Dr. Suparat Yeamkuan
ดร. ศุภรัตน์ แยมครวญ
suparat.y@bu.ac.th

School of Engineering
Artificial Intelligence Engineering and Data Science

Outline

- Dimensionality
- Dimensional Data Representation
- The Curse of Dimensionality
- High-dimensional Data
- Dimensionality Reduction
- Dimensionality Reduction Techniques
 - Linear Projection Methods (PCA, SVD, Random Projection)
 - Nonlinear and Manifold Learning (t-SNE, Isomap)
- Q&A

Dimensionality (มิติ)

มิติ (Dimensionality) หมายถึง จำนวนของคุณลักษณะ (features) ที่อยู่ในชุดข้อมูล (Dataset)

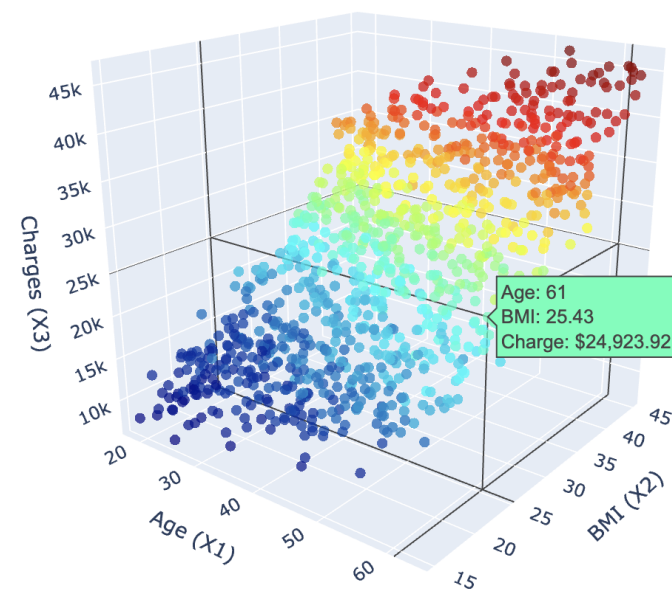
- มิติของแพตเทิร์น (pattern) หรือปริภูมิของแพตเทิร์น (pattern space) สามารถนิยามอย่างเป็นทางการได้ว่าเป็น จำนวนของ features (หรือแอตทริบิวต์) ที่ประกอบอยู่ใน pattern space นั้น
- ในเชิงคณิตศาสตร์ Dimensionality หมายถึง จำนวนของสมาชิกในชุดลำดับของค่าตัวเลข เช่น เวกเตอร์คุณลักษณะ (feature vector)

	Age	BMI	Charges
0	56	24.670615	20710.199440
1	46	39.296233	35862.766823
2	32	22.639220	18749.515655
3	60	35.445082	36282.830919
4	25	37.806836	27263.466753

Dataset with feature columns
Age, BMI, Charge

$$\mathbf{X} = \begin{matrix} & \begin{matrix} X1 & X2 & X2 \end{matrix} \\ \begin{bmatrix} 56 & 24.67 & 20710.20 \\ 46 & 39.30 & 35862.77 \\ 32 & 22.64 & 18749.52 \\ 60 & 35.45 & 36282.83 \\ 25 & 37.81 & 27263.47 \end{bmatrix} \end{matrix}$$

Dataset → Matrix
X1 (Age), X2 (BMI), X3 (Charge)



3 Dimensional Plot

Dimensional Data Representation

Mathematical Representation

ข้อมูลเชิงตารางแทนด้วยเมทริกซ์

$$\mathbf{X} \in \mathbb{R}^{n \times d}$$

n = จำนวนตัวอย่าง (samples)

d = จำนวนฟีเจอร์ (features)

ที่ ทุกค่าภายในเป็นจำนวนจริง $\mathbb{R}$

ตัวอย่าง:

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix}$$

High-dimensional Data

ข้อมูลมิติสูง (High-dimensional data) หมายถึงชุดข้อมูลที่มีจำนวนคุณลักษณะหรือแอตทริบิวต์ (d) จำนวนมาก เมื่อเปรียบเทียบกับจำนวนตัวอย่างหรือจำนวนข้อมูลสังเกต (n)

$$\mathbf{X} \in \mathbb{R}^{n \times d}, \quad \text{with } d \gg n$$

Type of High-dimensional Data

- ข้อมูลเชิงตาราง (Tabular Data)
- ข้อมูลภาพ (Image Data)
- ข้อมูลสัญญาณ (Signal Data) เช่น สัญญาณเสียง , Vibration / Acoustic signals
- ข้อมูลข้อความ (Text Data) แทนข้อความด้วยเวกเตอร์ Word Embeddings

High-dimensional Tabular Data

ข้อมูลเชิงตาราง (tabular) ทั่วไปแต่ละ column = 1 feature

กรณีศึกษา: ข้อมูลลูกค้าประกันสุขภาพ

สมมติว่ามีข้อมูลลูกค้า $n = 5$ คน
แต่มีคุณลักษณะ (features) จำนวนมาก $d = 8$ มิติ

ID	Age	BMI	BP	Chol	Glucose	Smoker	Region	Charges
1	56	24.7	130	210	98	1	2	20710
2	46	39.3	145	250	110	0	1	35862
3	32	22.6	120	190	90	0	3	18749
4	60	35.4	150	260	115	1	2	36282
5	25	37.8	140	230	105	0	4	27263

เลือกเฉพาะ features (ไม่รวม target Charges)
จะได้เมทริกซ์ข้อมูล

$$X \in \mathbb{R}^{n \times d} \rightarrow X \in \mathbb{R}^{5 \times 8}$$

ข้อมูลจะเรียกว่า High-dimensional data เมื่อ

$$d \gg n$$

หรือตัวอย่างอื่น เช่น

ลูกค้า 200 คน ($n = 200$)
คุณลักษณะ 5,000 ตัว ($d = 5000$)

$$X \in \mathbb{R}^{200 \times 5000}$$

High-dimensional Image Data

ภาพดิจิทัลสามารถมองเป็น ข้อมูลเชิงตาราง (tabular) ได้ โดยแต่ละ พิกเซล = 1 feature

ตัวอย่าง ภาพขาว-ดำ (Grayscale)

ภาพขนาด 28 x 28 (เช่น MNIST)

จำนวนพิกเซล = $28 \times 28 = 784$ โดย หนึ่งภาพ = เวกเตอร์มิติ 784

$$X \in \mathbb{R}^{n \times d} \rightarrow X \in \mathbb{R}^{784}$$

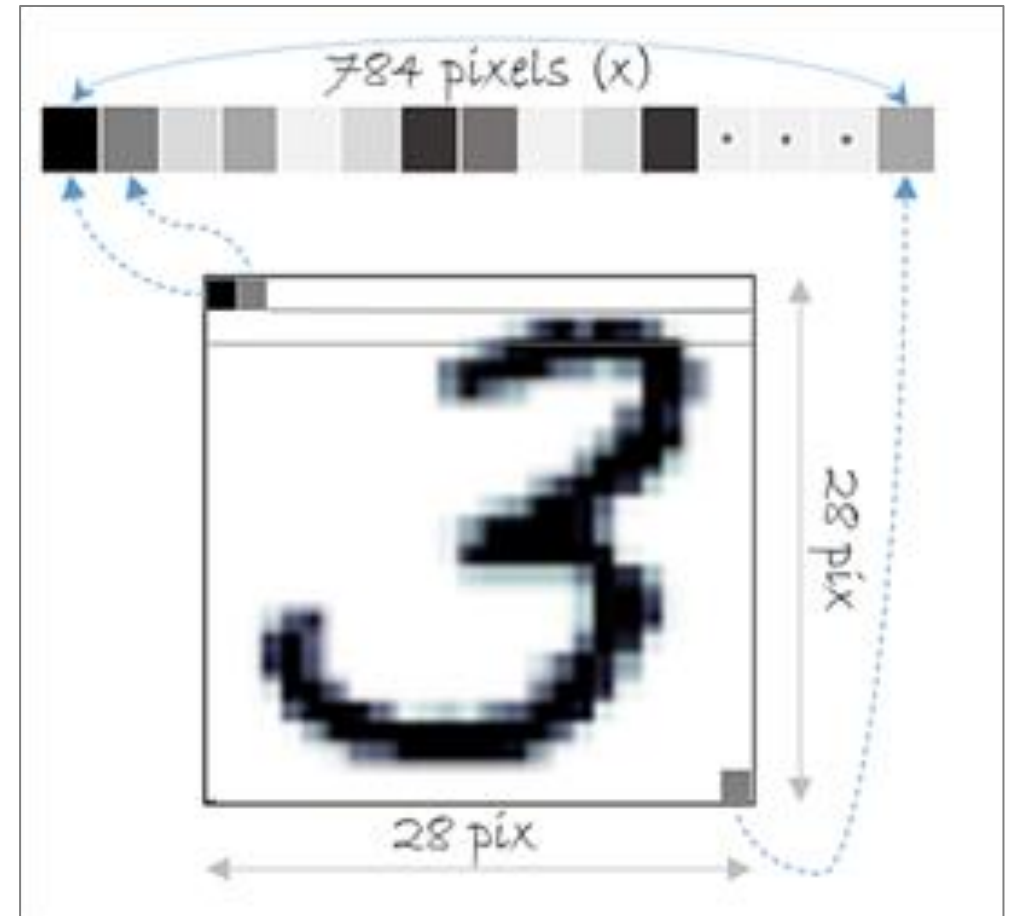
ถ้ามีภาพทั้งหมด n ภาพ

$$X \in \mathbb{R}^{n \times 784}$$

ตัวอย่างเชิงคณิตศาสตร์

$$\mathbf{x} = [d_1 \quad d_2 \quad \dots \quad d_{784}]$$

โดย d_i คือค่าความเข้มของพิกเซล (0–255 หรือ 0–1)



ตัวอย่าง รูปจาก MNIST

High-dimensional Signal Data

ตัวอย่างสัญญาณเสียง (Audio Signal)

Sampling rate = 44,100 Hz

ความยาวเสียง = 5 วินาที

จำนวนข้อมูล

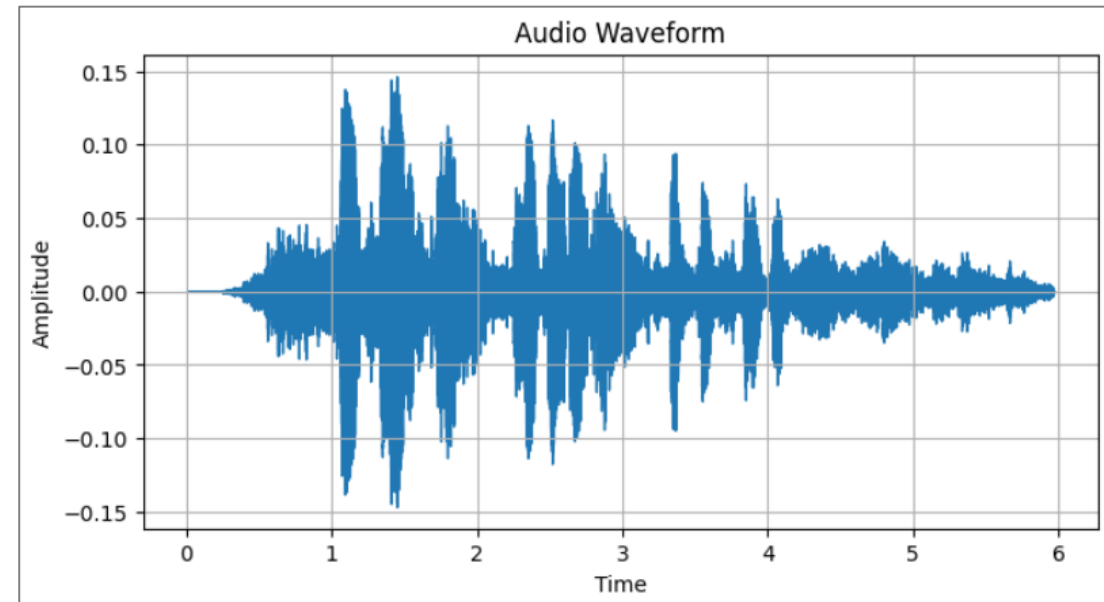
$$T = 44,100 \times 5 = 220,500$$

ดังนั้นเสียง **1 ไฟล์** แทนได้เป็น

$$\mathbf{x} \in \mathbb{R}^{220,500}$$

ถ้ามีเสียง n ไฟล์

$$\mathbf{X} \in \mathbb{R}^{n \times 220,500}$$



<https://www.geeksforgeeks.org/nlp/understand-audio-data/>

เมื่อจำนวนมิติของข้อมูล (features) เพิ่มขึ้นอย่างมาก

ข้อมูลจะเข้าสู่สภาวะที่เรียกว่า High-dimensional data ซึ่งนำไปสู่
ปัญหาพื้นฐานที่รู้จักกันในชื่อ

The Curse of Dimensionality



The Curse of Dimensionality

คำสาปของมิติ

Bellman (1961), the various challenges and paradoxes that arise when analyzing, modeling, and processing data in high-dimensional spaces.

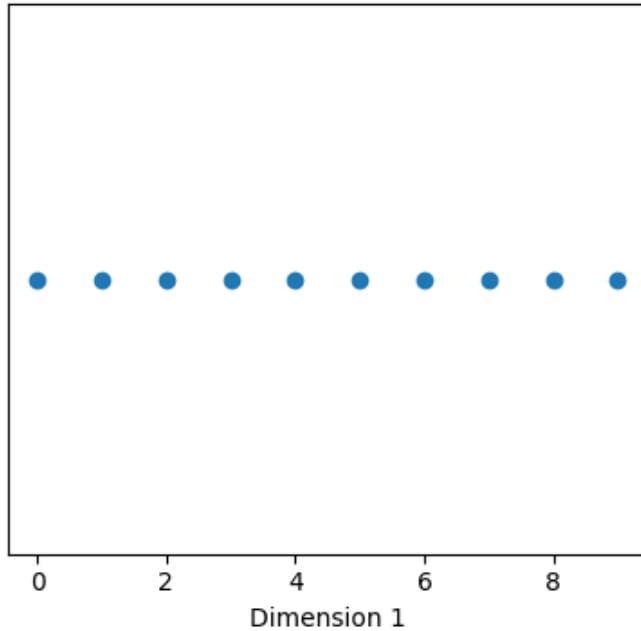
The curse of dimensionality is a common problem where the number of necessary data points is exponentially increased when the dimension is increased.

คำสาปของมิติ เป็นปัญหาที่พบได้บ่อย โดยเมื่อจำนวนมิติของข้อมูลเพิ่มขึ้น จำนวนข้อมูลที่ต้องใช้จะเพิ่มขึ้นแบบทวีคูณ (exponentially)

BOOK: Hands-On Machine Learning with TensorFlow.js
by Kai Sasaki

The Curse of Dimensionality in Points and Spaces

1D: 10 positions



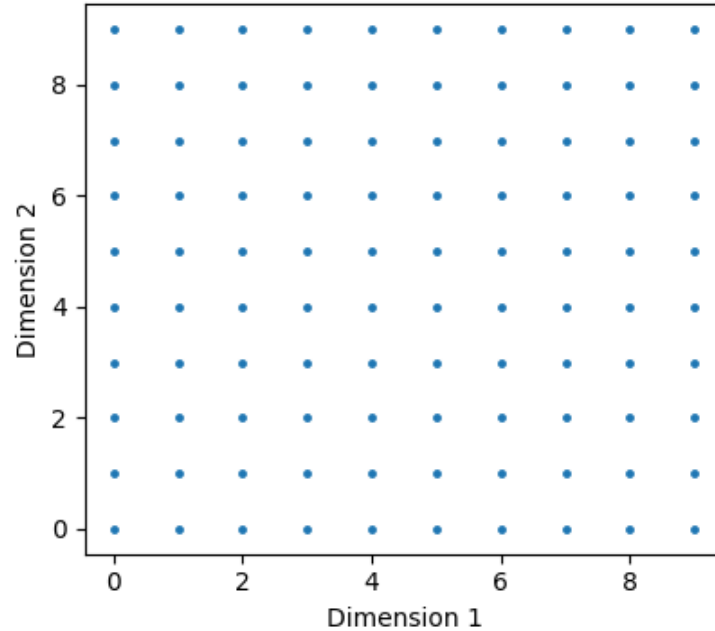
One Dimension (1D)

Total positions:

$$10^1 = 10$$

→ 10 Data Points

2D: $10 \times 10 = 100$ positions



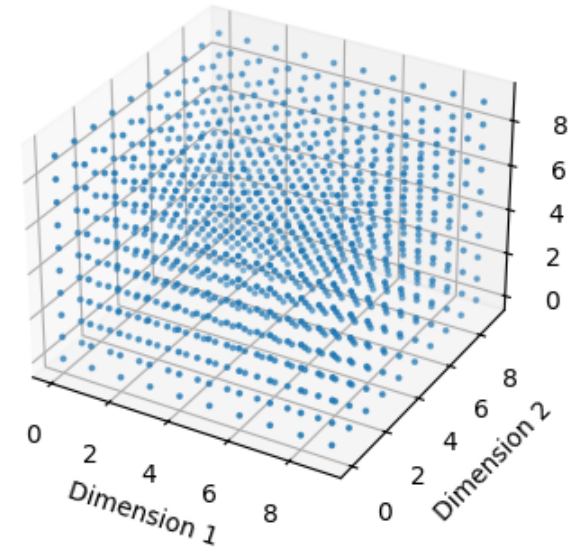
Two Dimension (2D)

Total positions:

$$10 \times 10 = 10^2 = 100$$

→ 100 Data Points

3D: $10 \times 10 \times 10 = 1000$ positions



Three Dimension (3D)

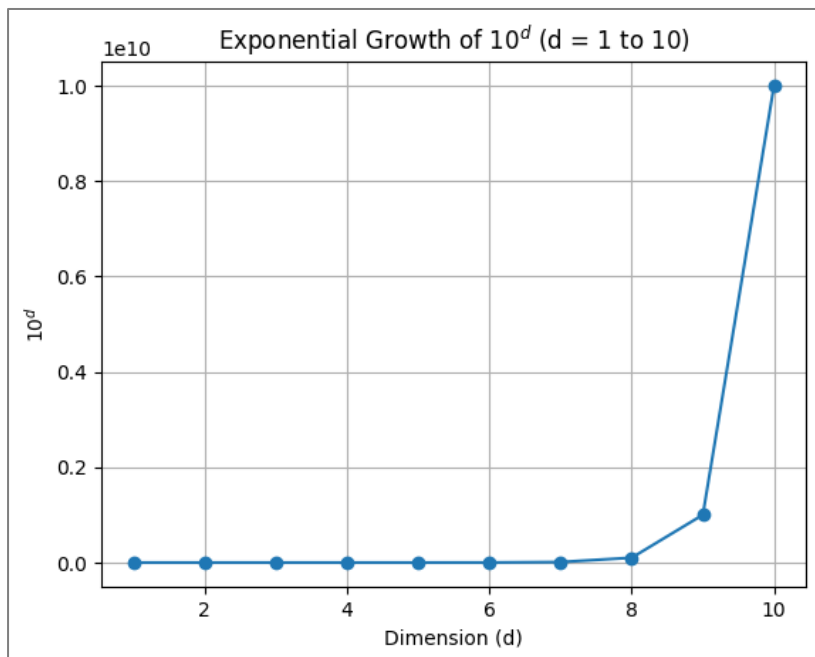
Total positions:

$$10 \times 10 \times 10 = 10^3 = 1000$$

→ 1000 Data Points

The Curse of Dimensionality

ในปริภูมิที่มีมิติสูง ปริมาตรของปริภูมิคุณลักษณะ (feature space) จะเพิ่มขึ้นแบบเอ็กซ์โพเนนเชียล (exponentially) เมื่อมีการเพิ่มมิติในแต่ละครั้ง



d	10^d
1	10
2	100
3	1,000
4	10,000
5	100,000
10	10,000,000,000

The exponential growth of a feature space's volume and the resulting data sparsity—is a fundamental challenge in machine learning that requires transforming high-dimensional data into more manageable representations

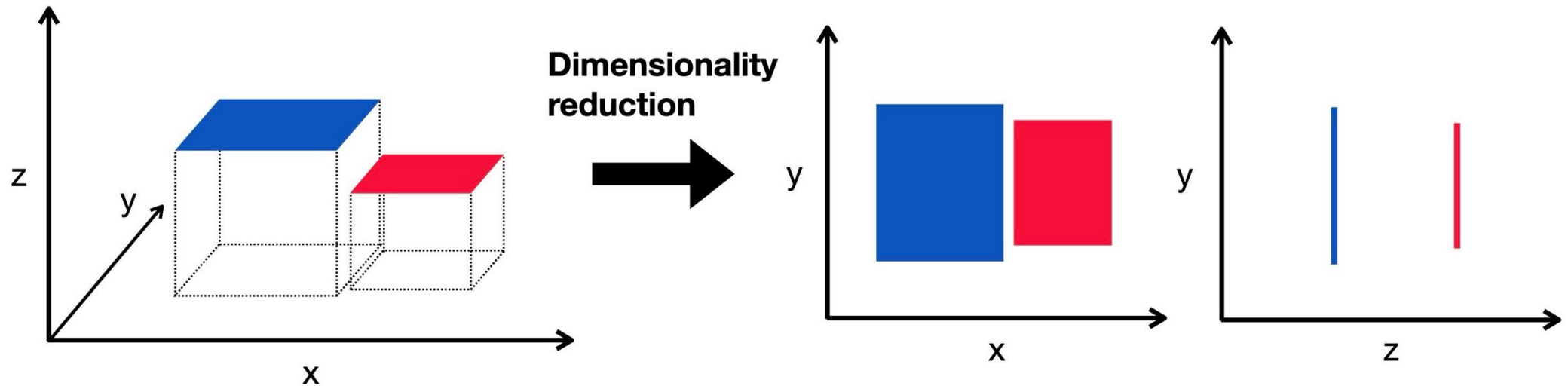
Overcome the Curse of Dimensionality

Overcome the Curse of Dimensionality หมายถึง การใช้เทคนิคและแนวทางต่าง ๆ เพื่อลดผลกระทบจากข้อมูลมิติสูง ซึ่งทำให้ข้อมูลเบาบาง การคำนวณซับซ้อน และโมเดลเกิด overfitting ได้ง่าย

Key Points

- **Dimensionality Reduction:** ลดจำนวนมิติของข้อมูล
- **Feature Selection:** เลือกเฉพาะฟีเจอร์ที่สำคัญ
- **Regularization:** ลดความซับซ้อนของโมเดล ป้องกัน overfitting
- **Manifold Hypothesis:** ใช้โครงสร้างมิติต่ำที่ซ่อนอยู่ในข้อมูล
- **Increase Data / Better Sampling:** เพิ่มข้อมูลหรือสุ่มอย่างมีประสิทธิภาพ

Dimensionality Reduction



การลดมิติ (Dimensionality Reduction) คือกระบวนการฝังข้อมูลที่มีมิติสูง (high-dimensional data) ลงในปริภูมิที่มีมิติต่ำกว่า โดยการแทนข้อมูลในมิติต่ำนี้ยังคงสามารถสะท้อนโครงสร้างพื้นฐาน (underlying structure) ของข้อมูลเดิมได้ ในขณะที่ใช้จำนวนมิติน้อยที่สุดเท่าที่จำเป็น

Dimensionality Reduction

Dimensionality reduction algorithms project high-dimensional data to a low-dimensional space, retaining as much of the salient information as possible while removing redundant information.

"hands on unsupervised learning using python by ankur a patel"

อัลกอริทึมการลดมิติ (Dimensionality Reduction) จะทำการฉายข้อมูลที่มีมิติสูงไปยังปริภูมิที่มีมิติต่ำกว่า โดยพยายามคงรักษาข้อมูลสำคัญ (salient information) ไว้ให้ได้มากที่สุด พร้อมทั้งตัดข้อมูลที่ซ้ำซ้อนหรือไม่จำเป็นออกไป

Motivation for Dimensionality Reduction

Project high-dimensional data to a low-dimensional space.

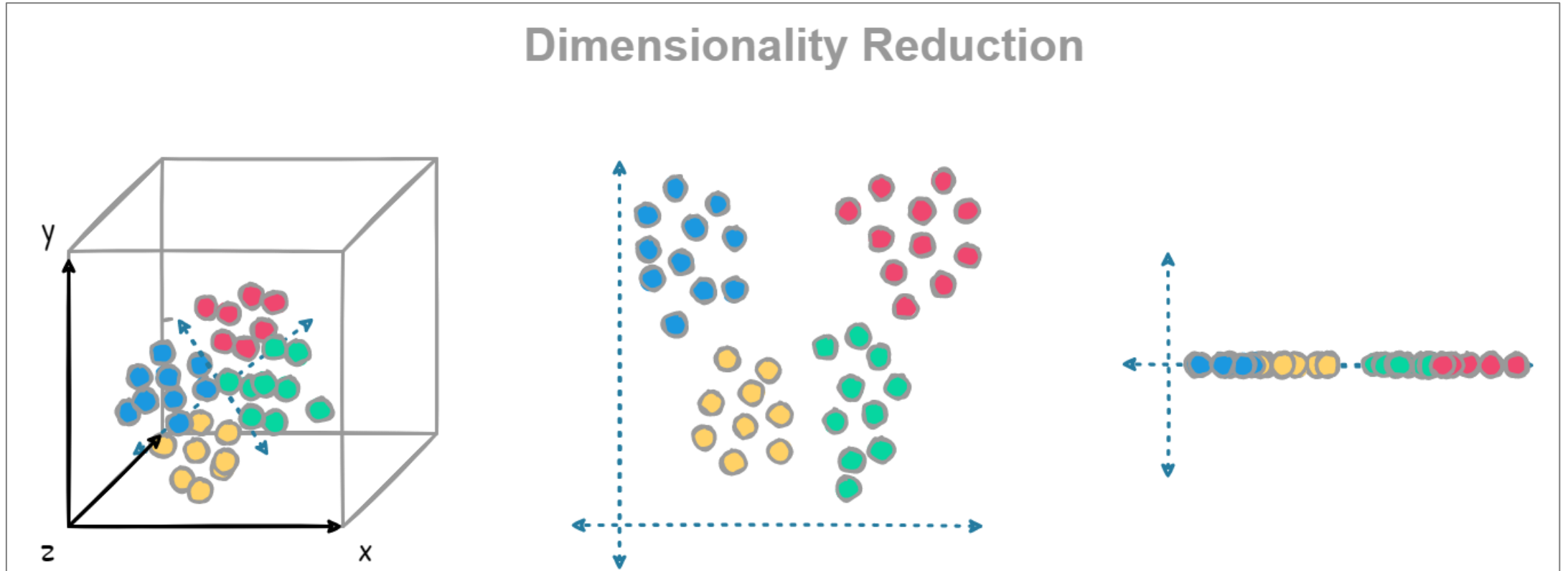


Image Source: <https://blog.roboflow.com/what-is-dimensionality-reduction/>

Feature engineering and dimensionality reduction

Feature Engineering → หากข้อมูลยังคง High-dimensional ต้องใช้ → Dimensionality Reduction เพื่อจัดการความซับซ้อนของข้อมูล

- **ตัวอย่างของการสร้างคุณลักษณะ (Feature Engineering) และการลดมิติ (Dimensionality Reduction)** คือ ดัชนีมวลกาย (Body Mass Index: BMI) ซึ่งมักใช้ในการระบุภาวะโรคอ้วน
- หากต้องการอธิบายภาวะโรคอ้วนอย่างละเอียด อาจพิจารณาคูณลักษณะหลายประการ เช่น ส่วนสูง น้ำหนัก เปอร์เซ็นต์ไขมันในร่างกาย ปริมาณกล้ามเนื้อ และเส้นรอบเอว
- อย่างไรก็ตาม BMI เป็นการรวมข้อมูลจากหลายตัวแปรให้เหลือเพียงค่าตัวเดียวที่ใช้งานได้ง่าย

สูตรดัชนีมวลกาย (Body Mass Index: BMI) คือ

$$\text{BMI} = \frac{\text{น้ำหนักตัว (กิโลกรัม)}}{(\text{ส่วนสูง (เมตร)})^2}$$

Dimensionality Reduction Techniques

Dimensionality Reduction Techniques

```
graph TD; A[Dimensionality Reduction Techniques] --> B[Linear Projection Methods]; A --> C[Nonlinear and Manifold Learning]; A --> D[Feature Selection];
```

Linear Projection Methods

- **Principal Component Analysis (PCA)**
- **Singular Value Decomposition (SVD)**
- **Random Projection**
- **Linear Discriminant Analysis (LDA)**

Nonlinear and Manifold Learning

- **Isomap**
- **t-SNE**
- **Locally Linear Embedding (LLE)**
- **Kernel PCA**
- **Multidimensional Scaling (MDS)**

Feature Selection

- **Filters**
- **Wrappers**
- **Embedded Methods**

Dimensionality Reduction in Unsupervised ML

Dimensional Reduction Techniques/Algorithms ที่นิยมใช้ร่วมกับ Unsupervised ML

Linear Projection Methods

- Random Projection
- Principal Component Analysis (PCA)
- Singular Value Decomposition (SVD)

Nonlinear and Manifold Learning

- t-Distributed Stochastic Neighbor Embedding (t-SNE)
- Manifold Learning Techniques (Isomap)

Linear Projection Methods

- Principal Component Analysis (PCA)
- Singular Value Decomposition (SVD)
- Random Projection

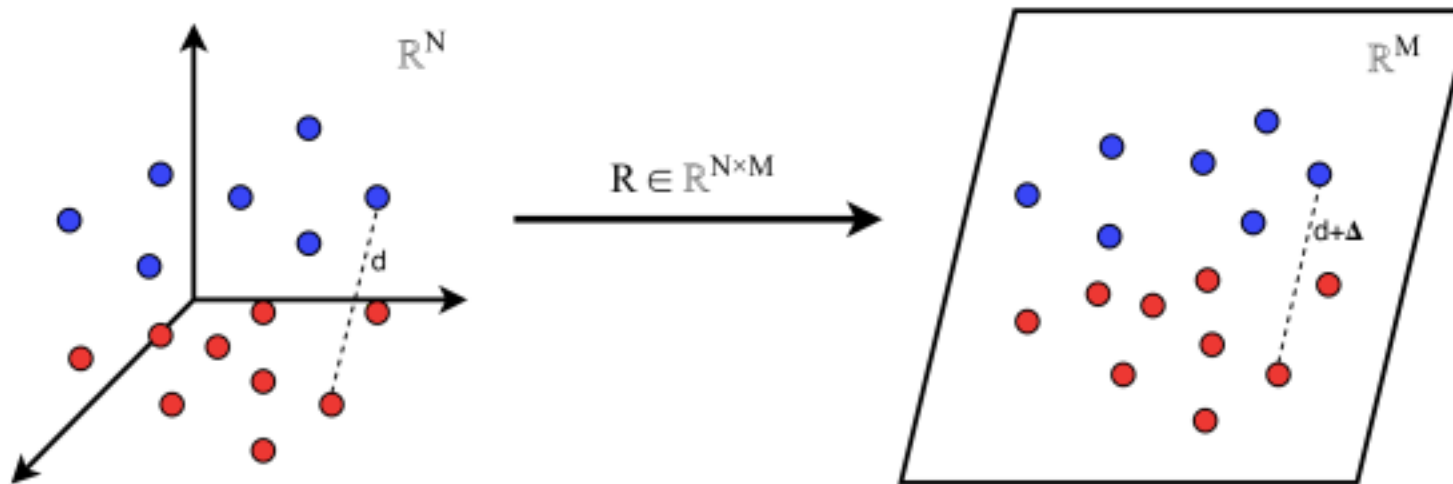




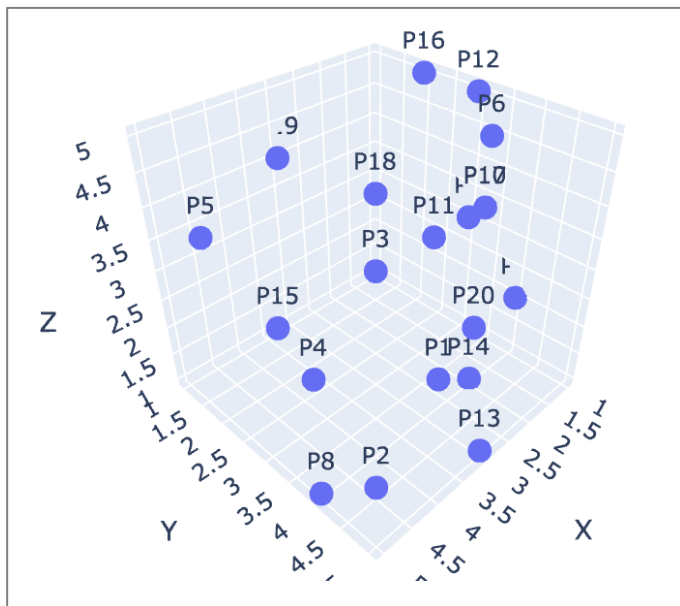
Random Projection

Random Projection

- **การฉายเชิงสุ่ม (Random Projection: RP)** เป็นเทคนิคการลดมิติแบบเชิงเส้น (**linear dimensionality reduction**) ซึ่งทำการฉายข้อมูลอินพุตที่มีมิติสูงดั้งเดิมลงไปยังปริภูมิย่อยที่มีมิติต่ำกว่ามาก
- เทคนิคนี้ถูกนำมาใช้เพื่อช่วยให้สามารถระบุรูปแบบของข้อมูลได้อย่างมีประสิทธิภาพมากขึ้น และช่วยแก้ปัญหามิติสูงที่มีต้นทุนการคำนวณสูง เช่น งานที่เกี่ยวข้องกับภาพ วิดีโอ เสียง และข้อความ



Understand Linear Projection



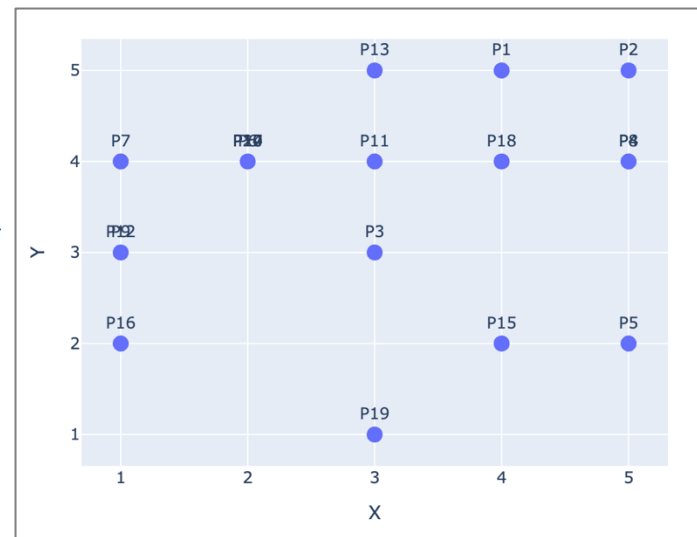
รูปแบบเมทริกซ์ (General form)

สมมติข้อมูลอยู่ในเมทริกซ์

$$X \in \mathbb{R}^{n \times 3}$$



$$Y = XW \in \mathbb{R}^{n \times 2}$$



เมทริกซ์ฉาย (projection matrix)

$$W \in \mathbb{R}^{3 \times 2}$$

Understand Linear Projection

Linear Projection ใช้ "Matrix Multiplication (การคูณเมทริกซ์)"

ตัวอย่างเชิงตัวเลข

$$X = \begin{bmatrix} 2 & 1 & 5 \\ 4 & 3 & 2 \\ 1 & 2 & 4 \end{bmatrix}$$

$$W = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

ผลลัพธ์ ข้อมูล 2 มิติ (X, Y)

$$Y = XW = \begin{bmatrix} 2 & 1 \\ 4 & 3 \\ 1 & 2 \end{bmatrix}$$

```
import numpy as np

# Data matrix X (3x3)
X = np.array([
    [2, 1, 5],
    [4, 3, 2],
    [1, 2, 4]
])

# Projection matrix W (3x2)
W = np.array([
    [1, 0],
    [0, 1],
    [0, 0]
])

# Linear projection
Y = np.dot(X, W)

print("\nProjected Data Y = XW:")
print(Y)
```

ทบทวนการคูณเมทริกซ์

$$\begin{bmatrix} a & b \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \end{bmatrix}$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} w & x \\ y & z \end{bmatrix} = \begin{bmatrix} aw + by & ax + bz \\ cw + dy & cx + dz \end{bmatrix}$$

Random Projection

Input

Dataset (4×3)

$$\mathbf{X} = \begin{bmatrix} 2 & 0 & 1 \\ 3 & 1 & 0 \\ 4 & 1 & 3 \\ 5 & 2 & 4 \end{bmatrix}$$

$$n = 4$$

$$d = 3$$

Step 1: Select dimension k

$$d = 3 \rightarrow k = 2$$

Step 2: Random Projection Matrix $\mathbf{R}$

$$\mathbf{R} \in \mathbb{R}^{3 \times 2} \rightarrow \begin{bmatrix} 0.6 & -0.4 \\ 0.1 & 0.8 \\ -0.2 & 0.5 \end{bmatrix}$$

Step 3: Projection

$$\mathbf{X}' = \mathbf{X}\mathbf{R} \rightarrow \mathbf{X}' = \begin{bmatrix} 2 & 0 & 1 \\ 3 & 1 & 0 \\ 4 & 1 & 3 \\ 5 & 2 & 4 \end{bmatrix} \begin{bmatrix} 0.6 & -0.4 \\ 0.1 & 0.8 \\ -0.2 & 0.5 \end{bmatrix}$$

Result of Random Projection

$$\mathbf{X}' = \begin{bmatrix} 1.0 & -0.3 \\ 1.9 & -0.4 \\ 1.9 & 0.7 \\ 2.4 & 1.6 \end{bmatrix}$$

Output

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.random_projection import GaussianRandomProjection

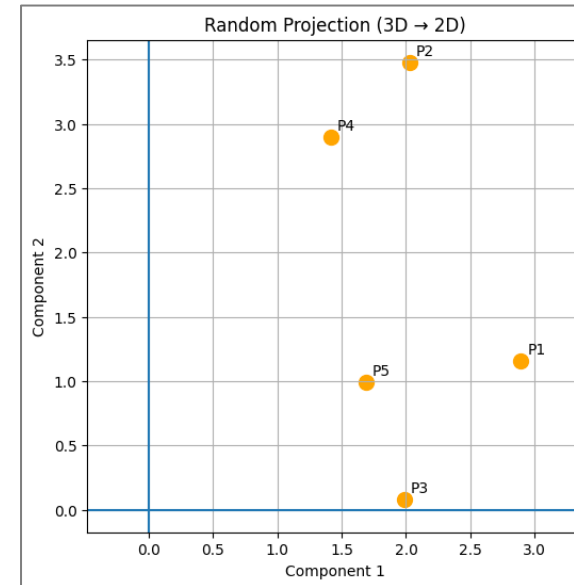
X = np.array([
    [2, 1, 5],
    [4, 3, 2],
    [1, 2, 4],
    [3, 1, 1],
    [2, 4, 3]
])

rp = GaussianRandomProjection(
    n_components=2,
    random_state=42
)

Y = rp.fit_transform(X)

W = rp.components_.T
print("Random Projection Matrix W:\n", W)
```

Random Projection Matrix $\mathbf{W}$:

$$\begin{bmatrix} 0.35122995 & 1.07694474 \\ -0.09776762 & -0.16557144 \\ 0.45798496 & -0.16555983 \end{bmatrix}$$


Real-Use Case of Random Projection

Problem:

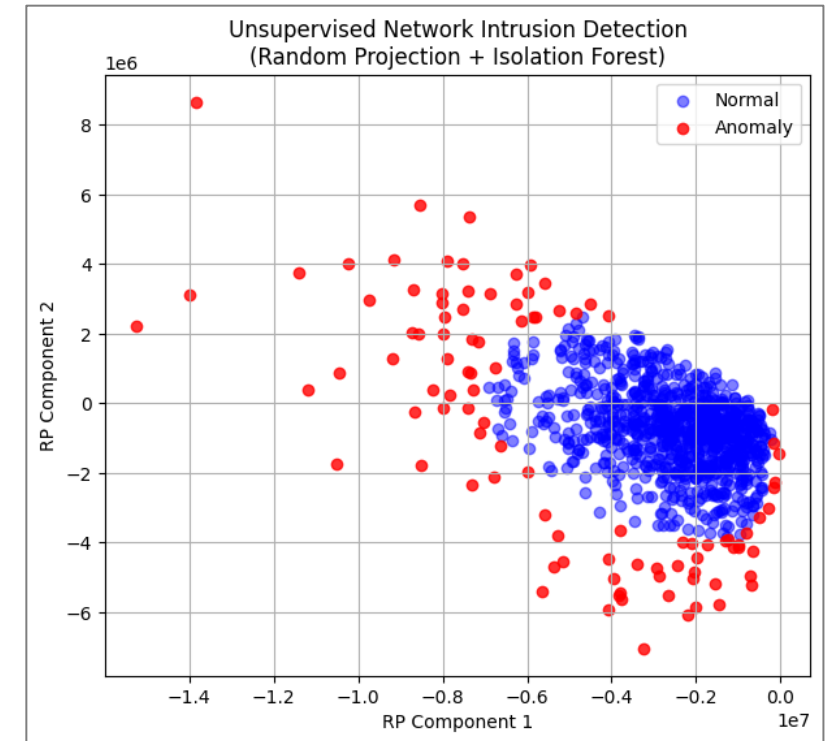
Detect and classify DDoS (Distributed Denial of Service) attacks

Protocol	Flow Duration	Total Fwd Packets	Total Backward Packets	Fwd Packets Length Total	Bwd Packets Length Total	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	Fwd Packet Length Std	Active Mean	Active Std	Active Max
6	1.197864e+06	10	6	273.393712	250.773177	1000.815139	20.002493	260.019686	66.975055	79258.911748	67110.257736	284555.698235
17	7.680784e+05	9	12	411.389735	76.981531	671.751416	35.561442	319.826738	106.740299	61547.527339	1397.768824	320180.168079
6	3.703793e+05	11	5	727.631806	287.045818	1031.993110	18.162610	335.968496	90.510932	40257.643145	40455.552776	614828.469935
6	1.680907e+06	11	7	465.282356	454.749796	698.739270	11.089000	427.465296	7.349375	107164.385553	9285.661597	317493.701756
6	1.154330e+06	13	7	323.643129	159.739552	527.055388	36.656517	438.276010	52.087047	26113.157123	49.617835	104717.408823

Network Flow (Original Feature Space) โดยสมมติ network flow แต่ละรายการเป็นเวกเตอร์คุณลักษณะ

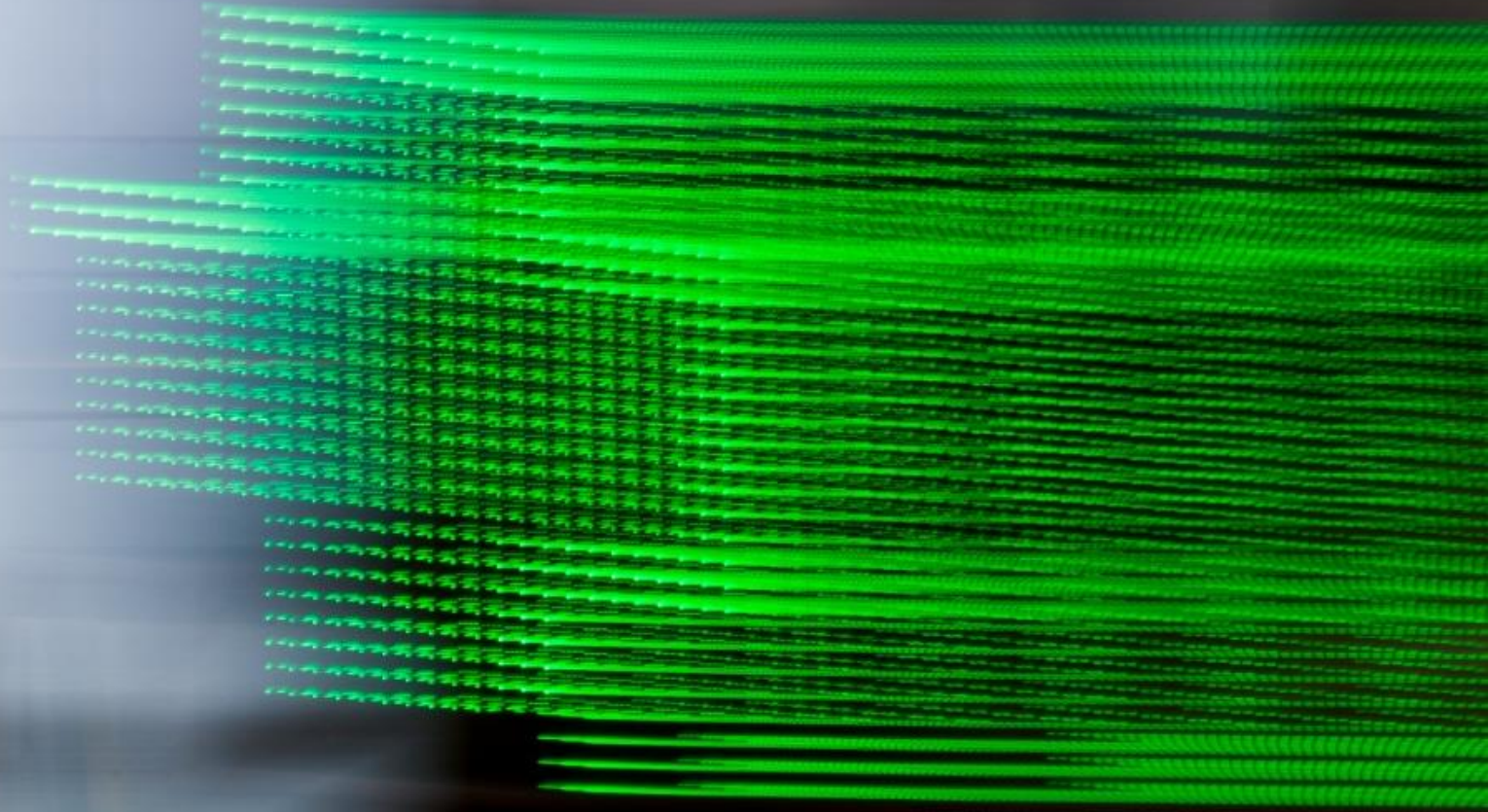
$$X \in \mathbb{R}^{n \times d} \rightarrow X \in \mathbb{R}^{300 \times 13}$$

Features: packet count, duration, bytes, flags, timing, etc.



Prediction Label	Anomaly	Normal
Normal	79	715
Syn	18	139
UDP_Flood	3	46

Principal Component Analysis (PCA)



Principal Component Analysis (PCA) Definition

การวิเคราะห์องค์ประกอบหลัก (Principal Component Analysis: PCA) เป็นเทคนิคการเรียนรู้ของเครื่องแบบไม่มีผู้สอน (unsupervised machine learning) ที่ได้รับความนิยม ใช้สำหรับการลดมิติแบบเชิงเส้น (linear dimensionality reduction) การบีบอัดข้อมูล (data compression) การสกัดคุณลักษณะ (feature extraction) และการแสดงผลข้อมูล (visualization)

Dataset (4 × 3)

$$\mathbf{X} = \begin{bmatrix} 2 & 0 & 1 \\ 3 & 1 & 0 \\ 4 & 1 & 3 \\ 5 & 2 & 4 \end{bmatrix}$$

$$n = 4$$

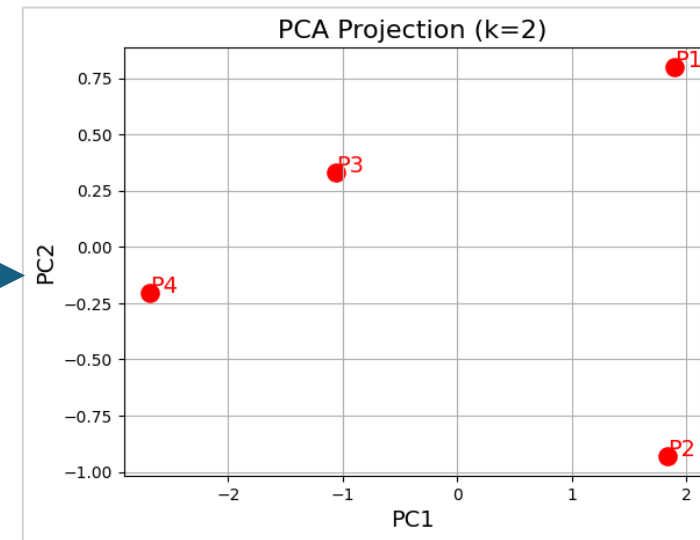
$$d = 3$$

PCA

PCA (4x2)

$$\mathbf{Z} = \begin{bmatrix} 1.899 & 0.799 \\ 1.840 & -0.930 \\ -1.056 & 0.333 \\ -2.683 & -0.203 \end{bmatrix}$$

Output



Principal Component Analysis (PCA)

Problem:

Plastic Injection Machine – Temperature Sensors → PCA (+ K-Mean)

เครื่องฉีดพลาสติกมี เซนเซอร์อุณหภูมิ 6 จุด โดยอุณหภูมิส่งผลต่อคุณภาพชิ้นงาน

$$X \in \mathbb{R}^{n \times d} \rightarrow X \in \mathbb{R}^{n \times 6}$$

สถานะสินค้าแบ่งเป็น 3 กลุ่ม:

PASS : คุณภาพดี

MIDDLE : กำกวม → ต้องใช้ Human Judgment

FAIL : ขอบเสีย

Principal Component Analysis (PCA)

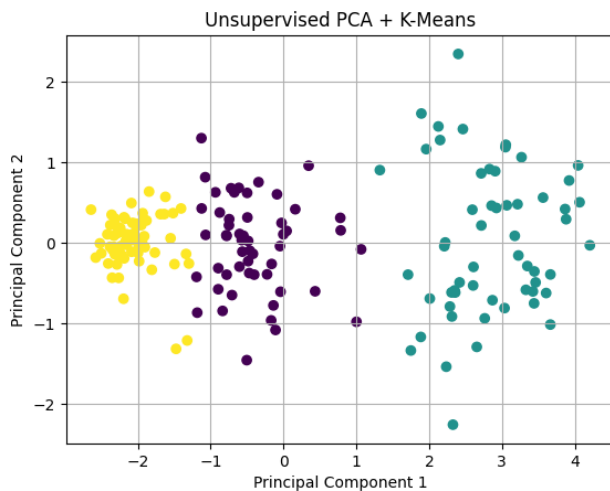
$X \in \mathbb{R}^{n \times 6}$ $\xrightarrow{\text{Standardize}}$ $\xrightarrow{\text{PCA}}$ $Z \in \mathbb{R}^{n \times 2}$ $\xrightarrow{\text{K-Means}}$ Clusters

	Temp_Sensor_1	Temp_Sensor_2	Temp_Sensor_3	Temp_Sensor_4	Temp_Sensor_5	Temp_Sensor_6
0	200.99	199.72	201.30	203.05	199.53	199.53
1	203.16	201.53	199.06	201.09	199.07	199.07
2	200.48	196.17	196.55	198.88	197.97	200.63
3	198.18	197.18	202.93	199.55	200.14	197.15
4	198.91	200.22	197.70	200.75	198.80	199.42
...	...	...	...	...	...	...
175	211.25	217.37	217.96	216.56	211.70	210.97
176	214.85	222.04	218.26	212.78	219.63	197.91
177	221.89	204.56	212.83	208.28	207.23	221.96
178	212.19	217.08	214.72	217.86	215.46	207.30
179	220.98	212.04	205.66	212.43	224.00	220.10

$X = \begin{bmatrix} 200.99 & 199.72 & 201.30 & 203.05 & 199.53 & 199.53 \\ 203.16 & 201.53 & 199.06 & 201.09 & 199.07 & 199.07 \\ 200.48 & 196.17 & 196.55 & 198.88 & 197.97 & 200.63 \\ 198.18 & 197.18 & 202.93 & 199.55 & 200.14 & 197.15 \\ 198.91 & 200.22 & 197.70 & 200.75 & 198.80 & 199.42 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 211.25 & 217.37 & 217.96 & 216.56 & 211.70 & 210.97 \\ 214.85 & 222.04 & 218.26 & 212.78 & 219.63 & 197.91 \\ 221.89 & 204.56 & 212.83 & 208.28 & 207.23 & 221.96 \\ 212.19 & 217.08 & 214.72 & 217.86 & 215.46 & 207.30 \\ 220.98 & 212.04 & 205.66 & 212.43 & 224.00 & 220.10 \end{bmatrix}$

PCA

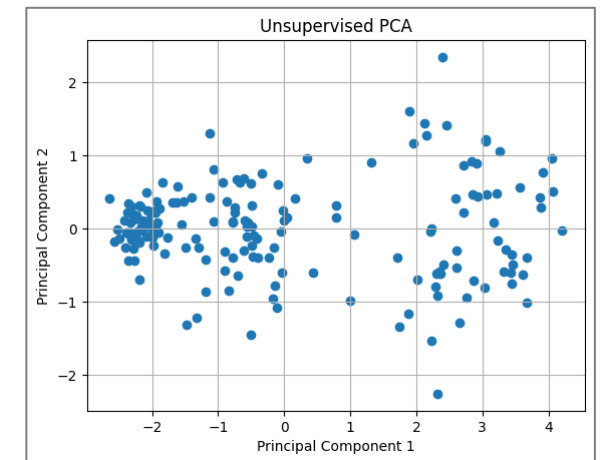
```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```



สถานะสินค้าแบ่งเป็น 3 กลุ่ม:
PASS : คุณภาพดี, MIDDLE : กำลัง, FAIL : ขอบเสีย

K-Means

```
kmeans = KMeans(
    n_clusters=3,
    random_state=42)
clusters = kmeans.fit_predict(X_pca)
```



Principal Component Analysis (PCA) Procedure

Dataset (4 × 3)

$$X = \begin{bmatrix} 2 & 0 & 1 \\ 3 & 1 & 0 \\ 4 & 1 & 3 \\ 5 & 2 & 4 \end{bmatrix}$$

$$n = 4 \\ d = 3$$

Input

Step 1: Mean of each feature (Column)

$$\mu_j = \frac{1}{n} \sum_{i=1}^n x_{ij} \rightarrow \mu = [3.5 \ 1 \ 2]$$

μ

Step 2: Mean-center the data

$$X_c = X - \mu \\ X_c = \begin{bmatrix} -1.5 & -1 & -1 \\ -0.5 & 0 & -2 \\ 0.5 & 0 & 1 \\ 1.5 & 1 & 2 \end{bmatrix}$$

X_c

Step 3: Covariance matrix

$$C = \frac{1}{n-1} X_c^T X_c = \frac{1}{3} \begin{bmatrix} 5 & 3 & 6 \\ 3 & 2 & 3 \\ 6 & 3 & 10 \end{bmatrix} \\ C = \begin{bmatrix} 1.667 & 1 & 2 \\ 1 & 0.667 & 1 \\ 2 & 1 & 3.333 \end{bmatrix}$$

$C \rightarrow A$

Define Eigenvalue and Eigenvector

$$A x = \lambda x$$

A = covariance matrix

x = eigenvector (principal component)

λ = eigenvalue (variance along that direction)

Step 4: Eigenvalue equation (k = 2)

$$\begin{bmatrix} 1.667 & 1 & 2 \\ 1 & 0.667 & 1 \\ 2 & 1 & 3.333 \end{bmatrix} x = \lambda x$$

→ Decomposition find x and λ

$$AX = X \Lambda$$

$X = [x_1 \ x_2 \ \dots \ x_d]$ = Eigenvectors

$\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_d)$ = Eigenvalues

Sort the eigenvalues
from largest → smallest

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$$

Eigenvectors sort as well

$$X = [x_1 \ x_2 \ \dots \ x_d]$$

X

Step 5: Select the first k = 2
Eigenvectors as columns

$$X_k = [x_1 \ x_2] ; (3 \times 2)$$

$$X_k = \begin{bmatrix} 0.54 & -0.62 \\ 0.32 & 0.77 \\ 0.78 & 0.14 \end{bmatrix}$$

X_k

Step 6: PCA projection

$$Z = X_c X_k$$

Z : data in 2D PCA space

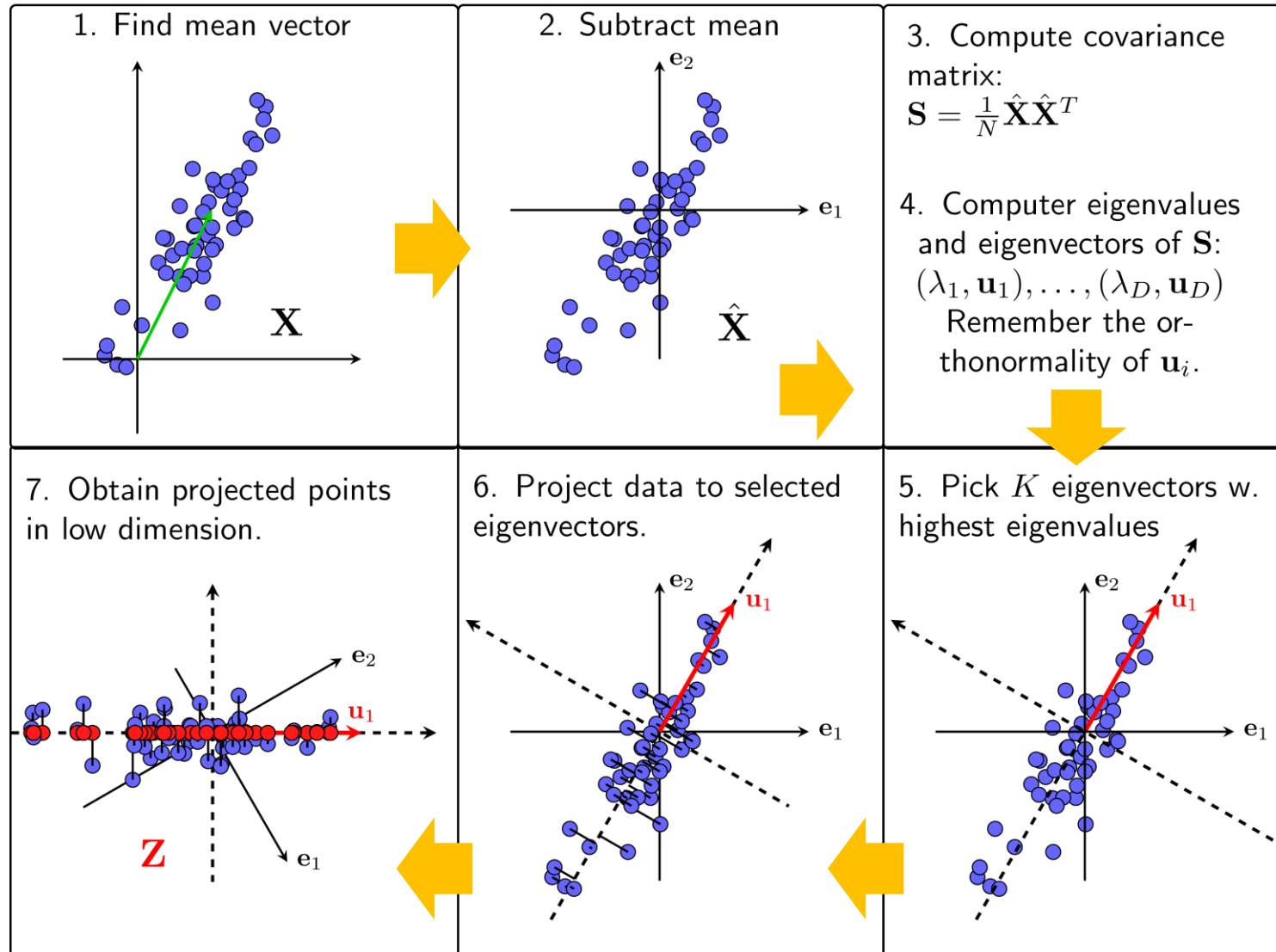
X_k : new axes

PCA (4x2)

$$Z = \begin{bmatrix} 1.899 & 0.799 \\ 1.840 & -0.930 \\ -1.056 & 0.333 \\ -2.683 & -0.203 \end{bmatrix}$$

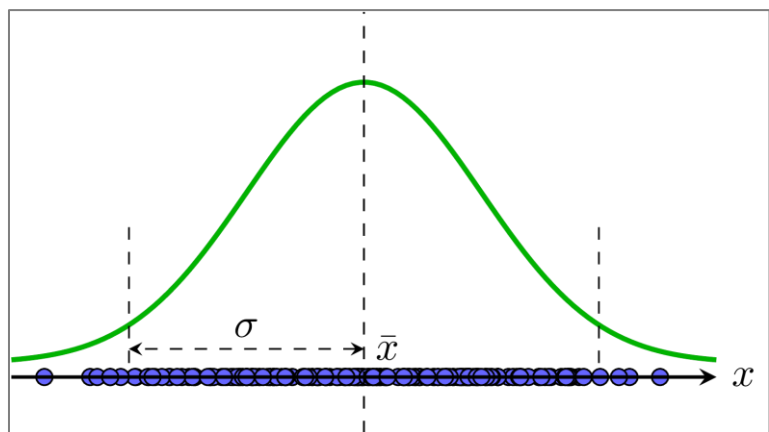
Output

PCA procedure

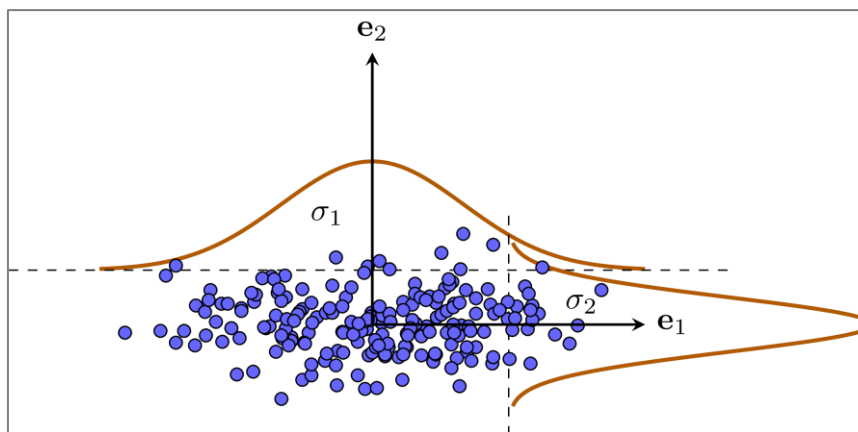


Principal Component Analysis (PCA)

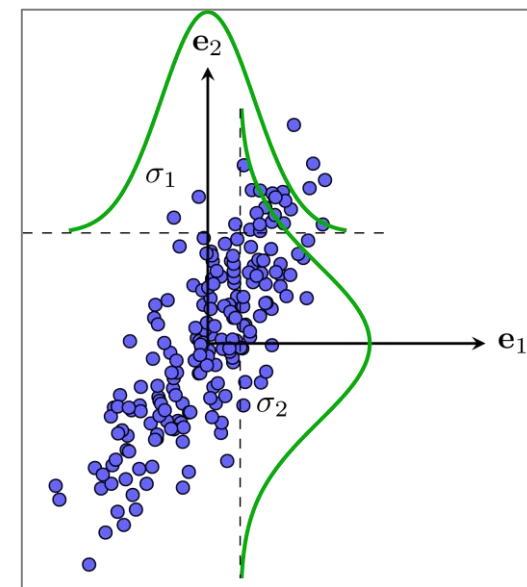
PCA คือการหมุนแกนข้อมูล (rotate axes) เพื่อหา eigenvectors ที่ทำให้ข้อมูลมีความแปรปรวน (variance) มากที่สุด และฉายข้อมูลลงบนแกนเหล่านั้นเพื่อลดมิติ (dimensionality reduction)



Examples of expectations and variances
1-dimensional space



Examples of expectations and variances
2-dimensional space where two dimensions are
not correlated



Examples of expectations and variances
Data in two-dimensional space is correlated



Singular Value Decomposition (SVD)

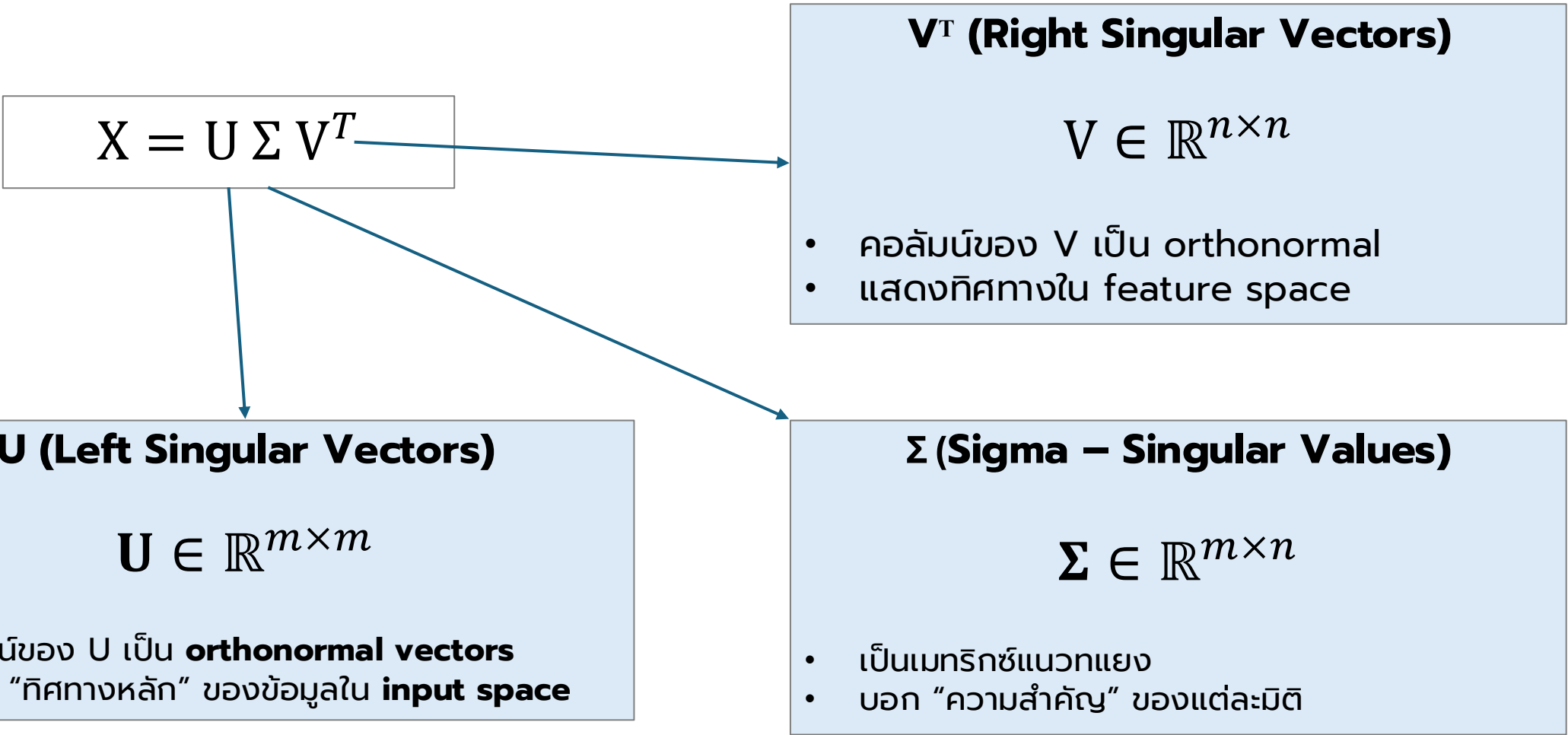
Singular Value Decomposition (SVD)

Singular Value Decomposition (SVD) คือ เทคนิคการแยกเมทริกซ์ (matrix factorization) ที่สำคัญมากในคณิตศาสตร์เชิงเส้นและ Machine Learning ใช้เยอะในงาน Dimensionality Reduction, Noise Reduction, Recommendation System, และ Image Compression

$$X = U \Sigma V^T$$

Matrix	ความหมาย
U	direction ของ data (sample space)
Σ	singular values (energy)
V	direction ของ features

องค์ประกอบของ SVD

$$X = U \Sigma V^T$$


V^T (Right Singular Vectors)

$$V \in \mathbb{R}^{n \times n}$$

- คอลัมน์ของ V เป็น orthonormal
- แสดงทิศทางใน feature space

U (Left Singular Vectors)

$$U \in \mathbb{R}^{m \times m}$$

- คอลัมน์ของ U เป็น **orthonormal vectors**
- แสดง "ทิศทางหลัก" ของข้อมูลใน **input space**

Σ (Sigma – Singular Values)

$$\Sigma \in \mathbb{R}^{m \times n}$$

- เป็นเมทริกซ์แนวทแยง
- บอก "ความสำคัญ" ของแต่ละมิติ

Singular Value Decomposition (SVD)

Dataset (4×3)

$$X = \begin{bmatrix} 2 & 0 & 1 \\ 3 & 1 & 0 \\ 4 & 1 & 3 \\ 5 & 2 & 4 \end{bmatrix}$$

$n = 4$
 $d = 3$

Input

4×3

Step 1: Find $X^T X$ (3×3)

$$X^T X = \begin{bmatrix} 54 & 16 & 34 \\ 16 & 6 & 11 \\ 34 & 11 & 26 \end{bmatrix}$$

Define Eigenvalue and Eigenvector

$$A v = \lambda v$$

A = covariance matrix

x = eigenvector

λ = eigenvalue

Step 2: Eigen decomposition $X^T X$

$$A v = \lambda v$$

Eigenvalues:

$$\lambda_1 \approx 82.15, \lambda_2 \approx 3.27, \lambda_3 \approx 0.58$$

Step 3: Singular Values

$$\sigma_i = \sqrt{\lambda_i}$$

$$\Sigma = \begin{bmatrix} 9.064 & 0 & 0 \\ 0 & 1.807 & 0 \\ 0 & 0 & 0.762 \end{bmatrix}$$

Inverse matrix of Σ

$$\Sigma^{-1} = \begin{bmatrix} 0.1103 & 0 & 0 \\ 0 & 0.5534 & 0 \\ 0 & 0 & 1.3123 \end{bmatrix}$$

Step 4: Find V the right Singular Vectors

Eigenvectors:

$$v_1, v_2, \dots, v_d$$

as

$$V = \begin{bmatrix} | & | & | \\ v_1 & v_2 & v_3 \\ | & | & | \end{bmatrix}$$

Eigenvectors A or $X^T X$ (3×3)

$$V = \begin{bmatrix} -0.804 & -0.527 & 0.276 \\ -0.257 & -0.112 & -0.960 \\ -0.537 & 0.842 & 0.046 \end{bmatrix}$$

Output (Z)

$$V_2 = \begin{bmatrix} -0.804 & -0.527 \\ -0.257 & -0.112 \\ -0.537 & 0.842 \end{bmatrix}$$

$$Z = X V_2$$

$$Z = \begin{bmatrix} -2.145 & -0.212 \\ -2.669 & -1.693 \\ -5.082 & 0.307 \\ -6.683 & 0.511 \end{bmatrix}$$

Step 6: Select $k = 2$

$$U_2 = [u_1 \ u_2]$$

Size:

$$U_2 \in \mathbb{R}^{4 \times 2}$$

$$U_2 = \begin{bmatrix} -0.237 & -0.117 \\ -0.294 & -0.937 \\ -0.561 & 0.170 \\ -0.737 & 0.283 \end{bmatrix}$$

Step 5: Find U

$$U = X V \Sigma^{-1}$$

$$U = \begin{bmatrix} -0.237 & -0.117 & 0.785 \\ -0.294 & -0.937 & -0.171 \\ -0.561 & 0.170 & 0.371 \\ -0.737 & 0.283 & -0.466 \end{bmatrix}$$

Problem:

Real-Use Case of SVD

ทำให้คอมเมนต์ที่ “พูดต่างคำ แต่ความหมายเหมือนกัน” อยู่ใกล้

```
comments = [  
    "The product is very good and worth the price",  
    "Good quality and very fast delivery",  
    "Very bad, completely unusable",  
    "Terrible product, broken right after opening the package",  
    "The service is okay, but the product is average",  
    "Slow delivery and poor packaging",  
    "Very worth it, affordable price",  
    "Poor quality, not worth the price"  
]
```

Text Vectorizer

	affordable	after	and	average	bad	broken	but
Comment_0	0.000	0.00	0.308	0.000	0.000	0.00	0.000
Comment_1	0.000	0.00	0.360	0.000	0.000	0.00	0.000
Comment_2	0.000	0.00	0.000	0.000	0.542	0.00	0.000
Comment_3	0.000	0.38	0.000	0.000	0.000	0.38	0.000
Comment_4	0.000	0.00	0.000	0.334	0.000	0.00	0.334
Comment_5	0.000	0.00	0.365	0.000	0.000	0.00	0.000
Comment_6	0.539	0.00	0.000	0.000	0.000	0.00	0.000
Comment_7	0.000	0.00	0.000	0.000	0.000	0.00	0.000

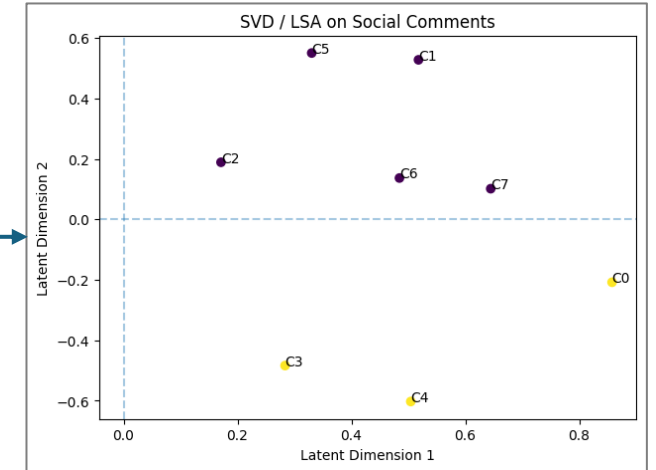
SVD

X_lsa

```
array([[ 0.85776427, -0.20784847],  
       [ 0.51779524,  0.52784096],  
       [ 0.17100817,  0.18891734],  
       [ 0.28352508, -0.48353437],  
       [ 0.50428554, -0.60240589],  
       [ 0.33030354,  0.55036862],  
       [ 0.48438388,  0.13686419],  
       [ 0.64465755,  0.10154265]])
```

Sentiment Score

```
--- Sentiment Score (latent dim 0) ---  
+0.858 : The product is very good and worth the price  
+0.518 : Good quality and very fast delivery  
+0.171 : Very bad, completely unusable  
+0.284 : Terrible product, broken right after opening the package  
+0.504 : The service is okay, but the product is average  
+0.330 : Slow delivery and poor packaging  
+0.484 : Very worth it, affordable price  
+0.645 : Poor quality, not worth the price
```



Adopted from: Multi-Class Classification with TF-IDF + SVD

<https://www.kaggle.com/code/seki32/multi-class-classification-with-tf-idf-svd/input>

Comparative Application Summary

Random Projection – PCA - SVD

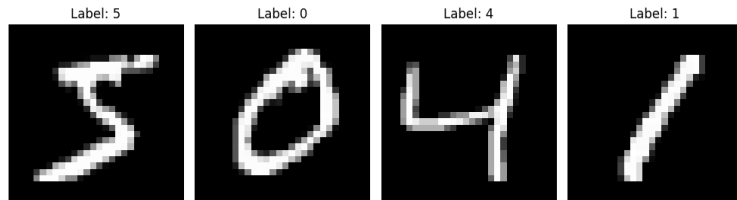
Feature	Principal Component Analysis (PCA)	Singular Value Decomposition (SVD)	Random Projection (RP)
Primary Goal	Maximize variance for representation.	Optimal low-rank matrix approximation.	Preserve distances at minimal cost.
Best For	Visualization and Denoising.	Recommender Systems and latent factors.	Big Data and Streaming data.
Efficiency	Expensive for huge datasets	Computationally heavy; similar to PCA.	Highly efficient and parallelizable.
Sparsity	Does not naturally preserve sparsity.	Effective for dense and sparse matrices.	Ideal for high-dimensional sparse vectors.

Nonlinear and Manifold Learning

- t-Distributed Stochastic Neighbor Embedding (t-SNE)
- Manifold Learning Techniques: Isomap (Isometric Mapping)



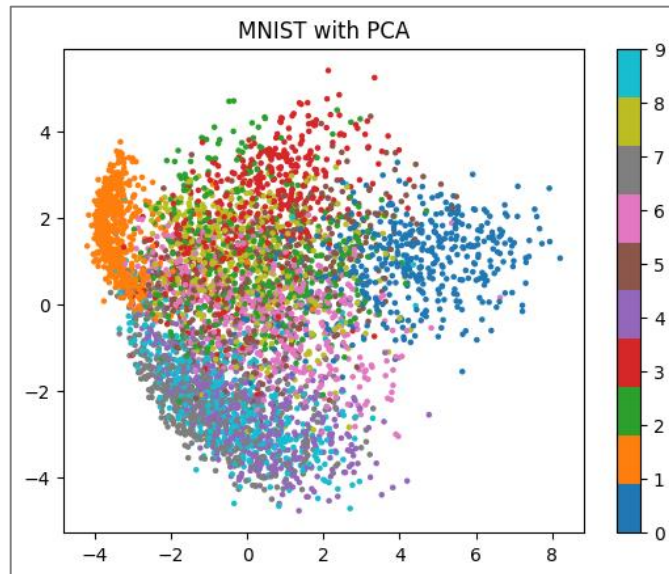
MNIST Dataset



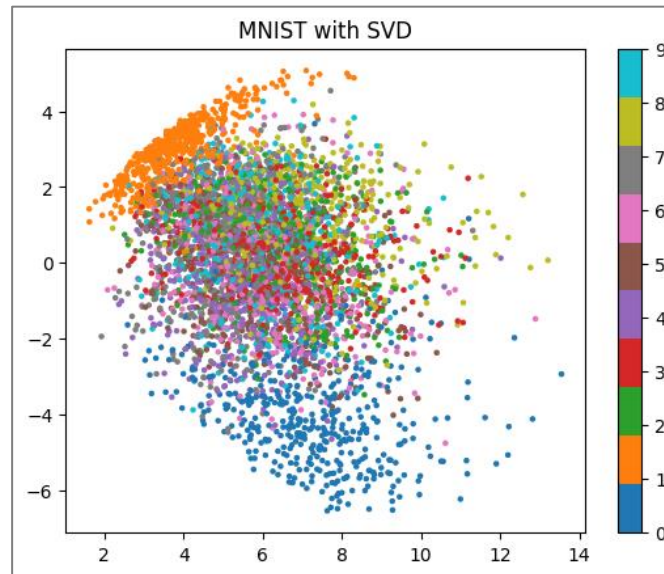
ลักษณะของ MNIST ข้อมูลเป็น ภาพ

โครงสร้างเป็น non-linear manifold
ตัวเลขเดียวกันอาจ เอียง เขียนหนา/บาง ตำแหน่งต่างกัน

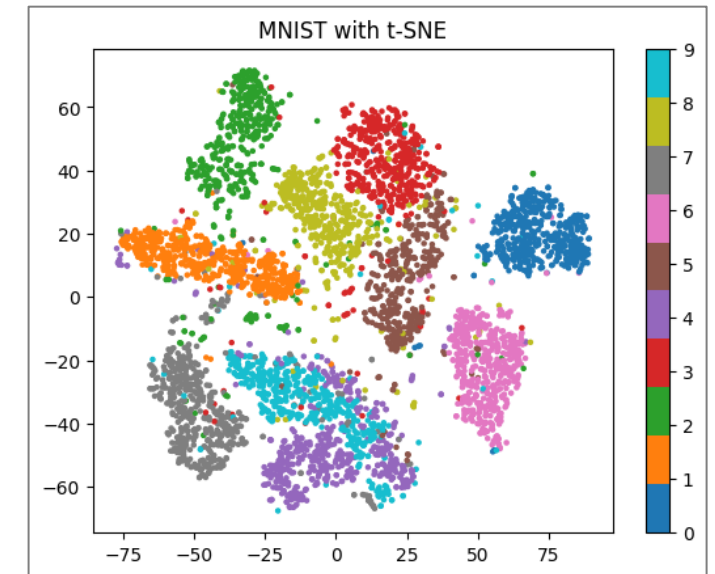
ความสัมพันธ์ระหว่างพิกเซล ไม่เป็นเชิงเส้น



PCA



SVD



t-SNE

t-Distributed Stochastic Neighbor Embedding (t-SNE)

- เป็นเทคนิค**การลดมิติแบบไม่เชิงเส้น**ที่ได้รับความนิยมอย่างมาก โดยถูกออกแบบมาโดยเฉพาะสำหรับการแสดงภาพข้อมูลที่มีมิติสูง (high-dimensional)
- เทคนิคนี้ทำงานโดยฝัง (embedding) จุดข้อมูลที่มีโครงสร้างซับซ้อนในมิติสูง ให้ไปอยู่ในปริภูมิที่มีเพียงสองหรือสามมิติ เพื่อให้สามารถมองเห็นและทำความเข้าใจโครงสร้างของข้อมูลได้ง่ายขึ้น

Real-Use Case of t-SNE

Problem:

การแบ่งกลุ่ม แรงจูงใจทางจิตวิทยา (psychological motivations) นักกีฬาระดับ Masters athletes

ลักษณะข้อมูล

- เป็น แบบสอบถามหลายข้อ
- แต่ละข้อ = 1 feature
- รวมกันเป็น high-dimensional psychological profile

คณิตศาสตร์:

$$\mathbf{X} \in \mathbb{R}^{n \times d}$$

n = จำนวนนักกีฬา

d = จำนวนคำถาม/ตัวแปรทางจิตวิทยา

ปัญหาหลักของข้อมูลจิตวิทยา

- ไม่เป็นเชิงเส้น (non-linear)
- ตัวแปรสัมพันธ์กันซับซ้อน
- กลุ่มแรงจูงใจทับซ้อนกัน

ตัวอย่างคำถาม

- Social affiliation → ความต้องการเป็นส่วนหนึ่งของสังคม
- Recognition → ความต้องการการยอมรับ
- Achievement → ความสำเร็จ
- Competition → การแข่งขัน
- Health / enjoyment → สุขภาพและความสนุก

Real-Use Case of t-SNE

Problem:

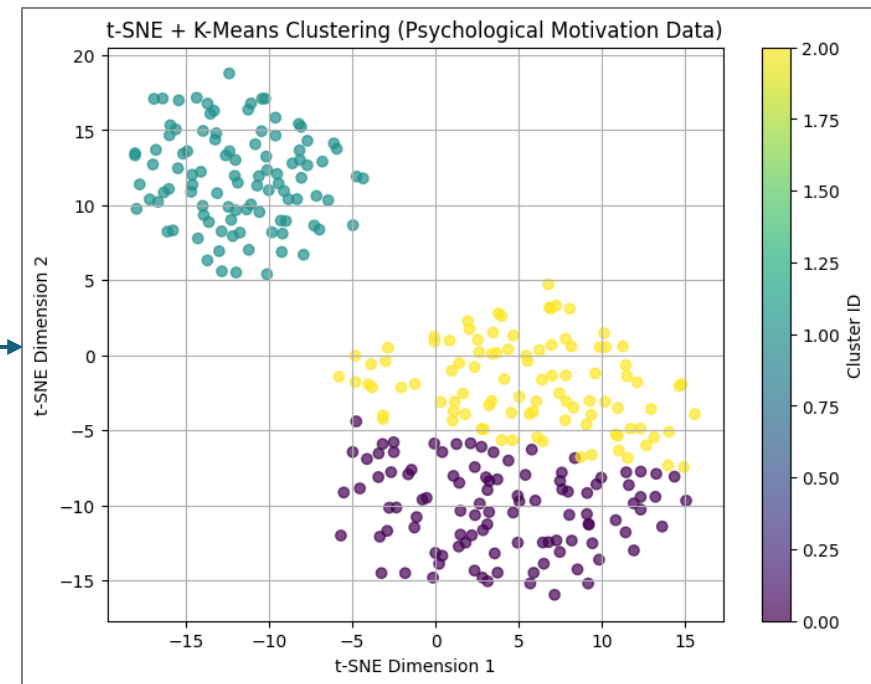
การแบ่งกลุ่ม แรงจูงใจทางจิตวิทยา (psychological motivations) นักกีฬาระดับ Masters athletes

	Social_Affiliation	Recognition	Achievement	Competition	Health	Enjoyment	Self_Esteem	Stress_Reduction
0	4.076291	4.593862	4.341933	5.142163	6.032960	5.699979	4.147946	5.790193
1	5.029712	3.677273	5.040727	5.758494	3.522633	2.434529	4.482711	4.434123
2	4.443778	5.029138	4.345881	2.326146	6.426707	5.534124	4.311126	3.601674
3	2.642263	5.080808	4.381461	4.663228	5.263970	3.990907	4.703582	4.591703
4	4.468350	3.691019	5.218835	3.397967	4.493846	4.150245	3.266324	3.011540

t-SNE

X_tsne

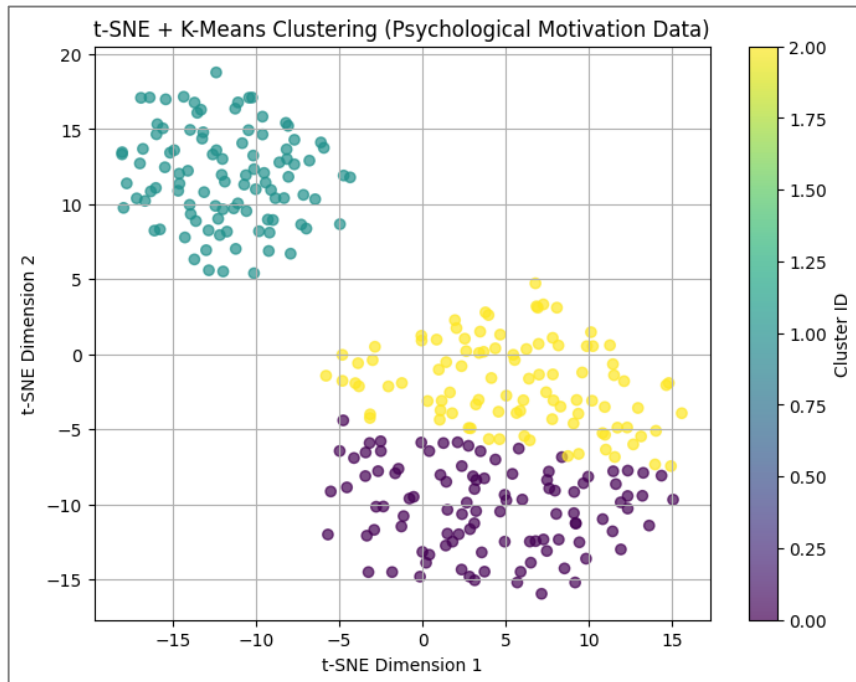
```
array([[ -1.39800558e+01,  1.49513941e+01],  
       [ -8.08271122e+00,  1.52208614e+01],  
       [ -1.41138487e+01,  1.22272663e+01],  
       [ -1.07284498e+01,  1.13061514e+01],  
       [ -1.13581543e+01,  9.72090435e+00],  
       [ -1.02455950e+01,  1.71068745e+01],  
       [ -1.20346985e+01,  1.30134306e+01],  
       [ -1.17663622e+01,  8.17112350e+00],  
       [ -1.64130650e+01,  1.71121311e+01],  
       [ -1.28727207e+01,  8.25560665e+00],
```



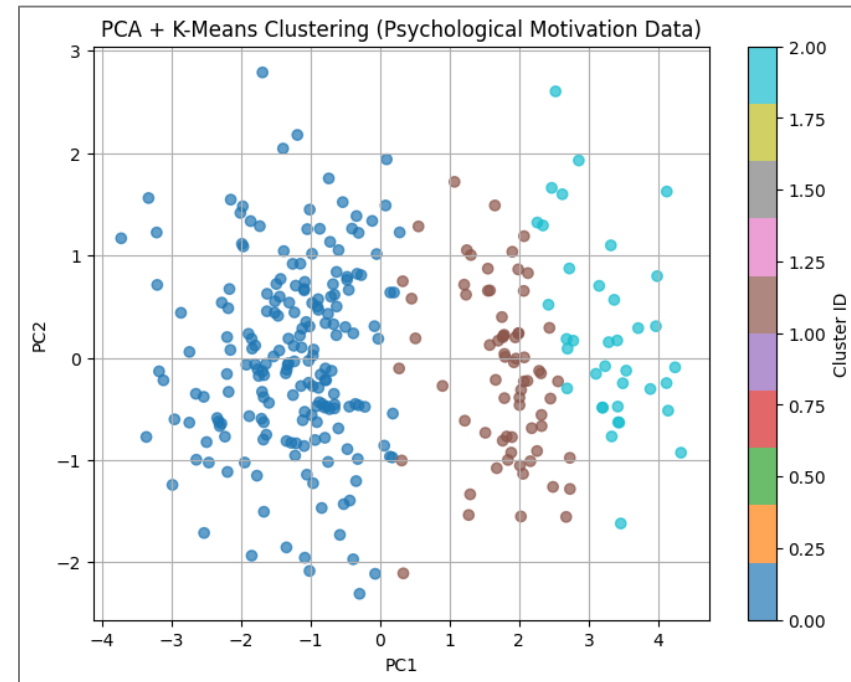
Real-Use Case of t-SNE

Problem:

การแบ่งกลุ่ม แรงจูงใจทางจิตวิทยา (psychological motivations) นักกีฬาระดับ Masters athletes



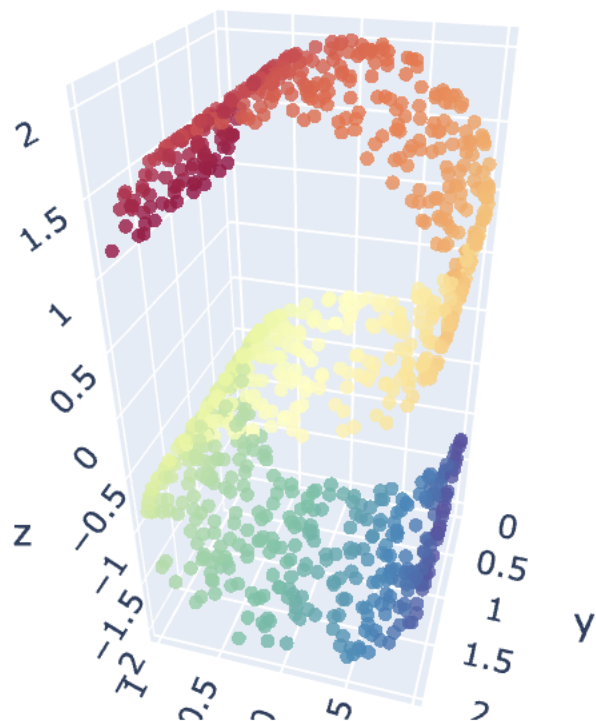
PCA



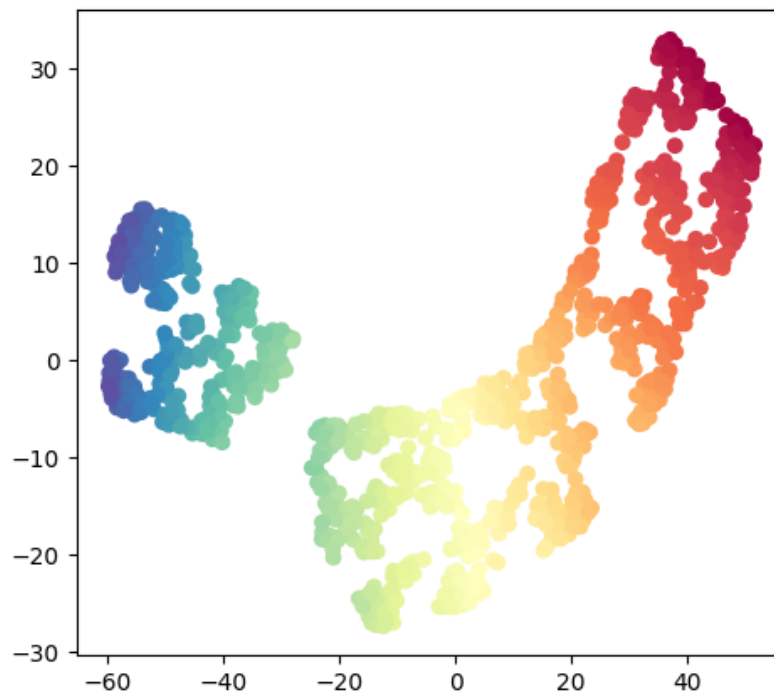
t-SNE

Clusters S-Shaped Dataset

Original 3D S-curve

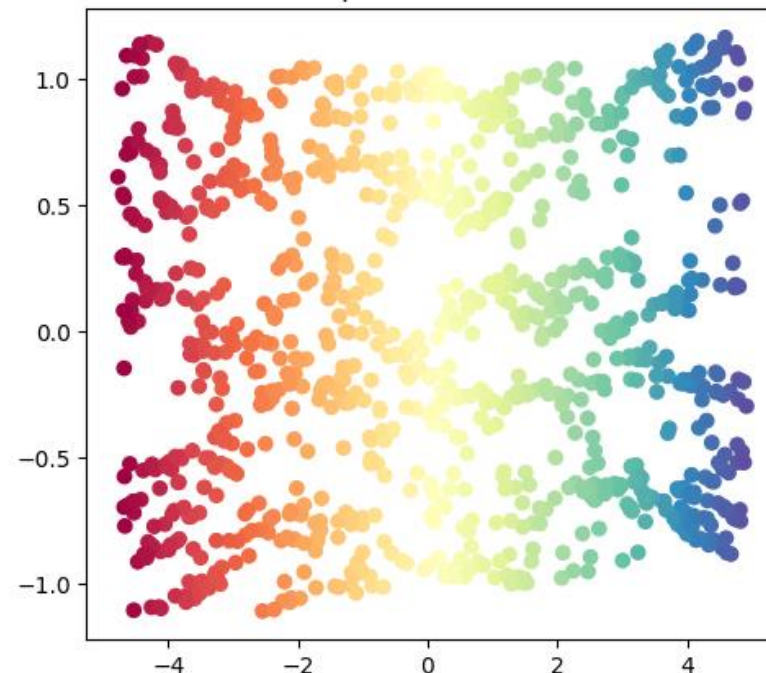


t-SNE Reduced 2D Data



t-SNE

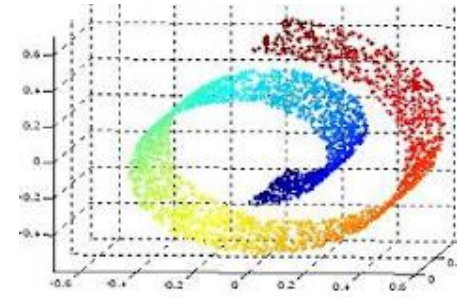
Isomap Reduced 2D Data



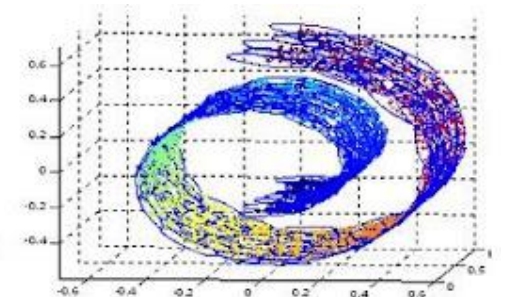
Isomap

Isometric Mapping: Isomap

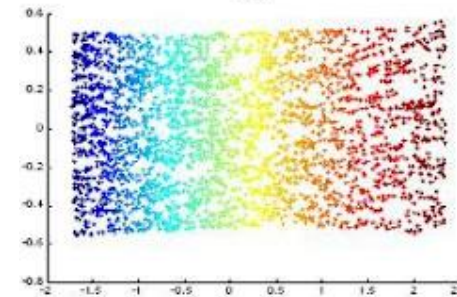
- Isomap คืออัลกอริทึม Dimensionality Reduction แบบ non-linear ในกลุ่ม Manifold Learning
- ใช้ลดมิติข้อมูล โดยพยายาม รักษาโครงสร้างเชิงเรขาคณิตของข้อมูลบนแมนิโฟลด์ (manifold) ไว้ให้มากที่สุด



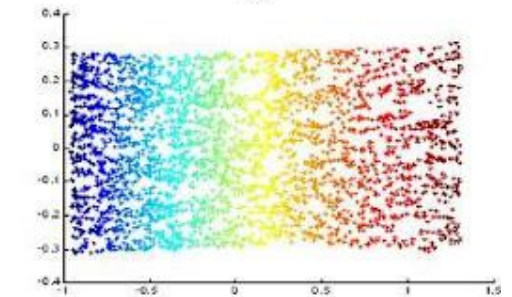
(a)



(b)



(c)



(d)

Real-Use Case of Isomap

Problem:

SOMAP Analysis of Sea Surface Temperature (SST) ^{El Niño} Classification & Detection of El Niño / La Niña

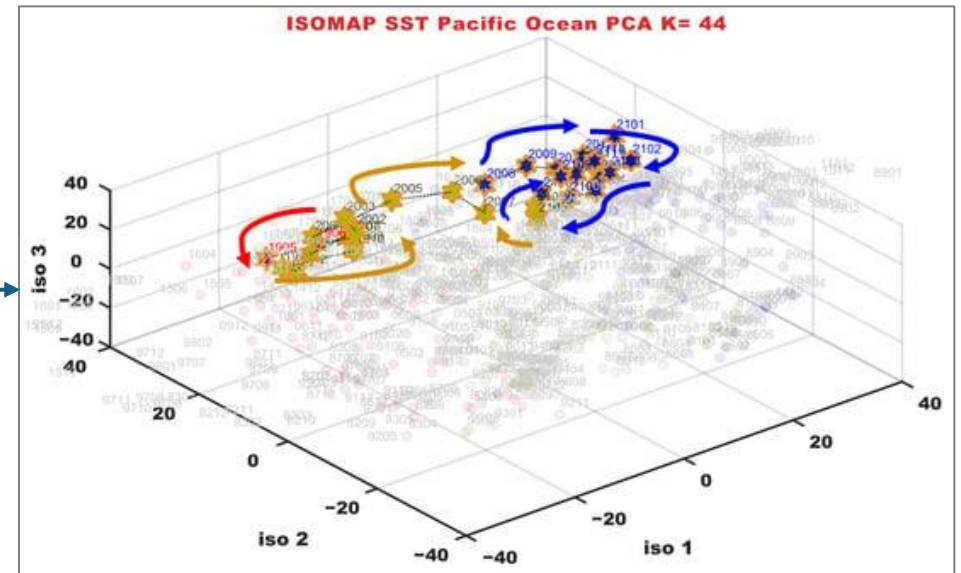
	SST_0	SST_1	SST_2	SST_3	SST_4	SST_5	SST_6	SST_7	SST_8	SST_9	...	SST_141	SST_142
0	27.248357	26.930868	27.323844	27.761515	26.882923	26.882932	27.789606	27.383717	26.765263	27.271280	...	27.653571	26.196258
1	27.125246	27.173224	26.659988	27.116127	27.146536	26.642824	27.932887	27.236916	26.404348	27.328277	...	26.753500	26.705318
2	26.585502	26.719909	27.373647	27.305185	26.989549	27.058664	27.638832	26.704214	27.273549	26.898904	...	26.983653	25.966279
3	26.968660	27.477571	26.507137	27.252023	26.734871	26.603564	26.946485	26.482379	26.723175	26.401061	...	26.761171	27.239490
4	27.378494	26.538917	27.434803	27.677819	27.206717	27.938398	26.613105	26.377673	26.110640	27.748022	...	26.360212	26.687591

5 rows x 151 columns

Isomap

X_iso

```
array([[ 0.67599409, -5.52514762],  
       [-5.25533593,  3.40203886],  
       [-5.84587546, -3.86875722],  
       [-5.83222538, -3.59522127],  
       [-6.2482869 , -3.56066248],  
       [-6.3242233 , -3.84432479],  
       [-5.88341156, -3.39666293],
```





SCHOOL OF
ENGINEERING
BANGKOK UNIVERSITY

School of Engineering
Artificial Intelligence Engineering and Data Science

Q & A